

راهنمای همراه نوت‌بوک

تمرین پایانی ۱۳: از شمارش کلمات تا ترنسفورمر

From Bag of Words to Transformer

این برگه چیست؟

کار اصلی‌ات داخل نوت‌بوک `week13_hw_projects.ipynb` است. کد را خودت بنویس. از کلاس ایده بگیر، ولی سلول آماده را کامل کپی نکن. مدل را کوچک نگه دار تا روی CPU جلو برود؛ مثلاً `d_model` حدود 32 تا 64 و یکی دو لایه کافی است.

به صورت کلی

- دو پروژه جدا داری. هر دو از متن خام شروع می‌شوند، ولی سؤال‌شان فرق می‌کند.
- پروژه ۱ مثل یک خبرخوان کوچک است: متن را می‌خواند و می‌گوید موضوع فضا است یا هاک. این طبقه‌بندی نظارت‌شده است؛ یعنی لیبل موضوع را از قبل داریم.
 - پروژه ۲ مثل یک قلم کمکی خیلی کوچک است: یک تکه متن می‌بیند و حدس می‌زند بعدی چیست. این مدل زبانی خودنظارتی است؛ لیبل را انسان نساخته، از خود متن آمده است.
- مسیر را از روی نوت‌بوک برو: اول پروژه ۱ تا آخر، بعد پروژه ۲.

چند اصطلاح که مدام می‌آیند

اگر وسط کار گیر کردی، برگرد این‌جا.

یعنی چه؟

- **Bag of Words / شمارش کلمات:** فقط می‌شمارد هر کلمه چند بار آمده. ترتیب را نگه نمی‌دارد. `dog bites cat` و `cat bites dog` برایش خیلی شبیه می‌شوند.
- **شناسه / ID:** یک شماره در واژه‌نامه. خودِ عدد معنی ندارد؛ فقط آدرس ردیف `embedding` است.
- **embedding:** به جای شماره، یک بردار متراکم می‌گذاریم که مدل می‌تواند یاد بگیرد کلمه‌های شبیه، نزدیک هم باشند.
- **skip-gram:** از روی کلمه وسط، کلمه‌های اطرافش را حدس می‌زنیم. سیگنالش این است که «همسایه‌ها شبیه هم‌اند».
- **اطلاعات موقعیت:** توجه به تنهایی ترتیب «اول / بعدی» ندارد. یک بردار موقعیت به هر توکن می‌دهیم تا مدل بفهمد کجای جمله است.
- **ماسک علی / causal mask:** پرده‌ای روی توجه که نگاه به توکن‌های بعدی را می‌بندد. `True` یعنی «این نگاه ممنوع است».
- **خودنظارتی / self-supervised:** هدف پیش‌بینی از خود داده ساخته می‌شود؛ مثلاً توکن بعدی. لازم نیست کسی موضوع را لیبل بزند.
- **LayerNorm:** ویژگی‌های همان یک توکن را پایدار می‌کند، نه کل دسته را.
- **logits:** امتیاز خام مدل برای هر عضو واژه‌نامه، قبل از `softmax`.

۱ پروژه ۱. خبرخوان کوچک

در نوت‌بوک از عنوان «پروژه ۱» تا پایان بخش ۱هـ.

اینجا چه می‌خواهیم؟

می‌خواهیم ببینیم هر بازنمایی چه چیزی از خبر می‌فهمد. آخر کار باید بتوانی با زبان خودت بگویی BoW چه چیزی را از دست می‌دهد، embedding چه چیزی اضافه می‌کند، و Encoder با موقعیت و توجه چه کار تازه‌ای می‌کند.

فرض کن برای یک اپ خبری کار می‌کنی. فعلاً فقط دو موضوع داری: فضا و هاکی. داده 20 Newsgroups واقعی و معروف است؛ برای اینکه تمرین سبک بماند همان دو کلاس sci.space و rec.sport.hockey را با حجم کم بردار (حدود ۲۵۰ آموزش و ۱۰۰ آزمون). اگر هدر ایمیل را حذف کنی، مدل بیشتر خودِ خبر را می‌بیند نه مشخصات فرستنده را.

بخش ۱الف. داده و تمیزکاری

متن وب و گروه خبری شلوغ است: لینک، علائم، حروف بزرگ، فاصله اضافه. نرمال‌سازی یعنی قبل از مدل، شکل متن را یکی کنیم تا دو نوشته تقریباً یکسان، دو چیز مختلف به نظر نرسند. وقتی نمونه خام را کنار نسخه تمیز می‌گذاری، باید بتوانی بگویی چه چیزی حذف شد و چرا حذفش منطقی است. لینک معمولاً موضوع خبر نیست؛ نویز است.

یک نکته کنار دستت

بارگذاری را با scikit-learn انجام بده و نام کلاس‌ها را دقیق بنویس. اگر تعداد دو کلاس کمی فرق داشت اشکالی ندارد؛ همان را در توضیح بنویس.

بخش ۱ب. خط پایه شمارش کلمات

اول یک مدل ساده می‌سازیم تا بعداً بفهمیم مدل بعدی واقعاً چه چیزی اضافه کرده. BoW یا TF-IDF هر دو برای این خط پایه خوب‌اند. یک رگرسیون لجستیک روی این بردار کافی است. دقت خیلی بالا لازم نیست. جالب‌تر این است که دو خبر اشتباه را بخوانی و حدس بزنی چرا. بعد دو جمله از خودت بساز که معنی‌شان فرق دارد ولی تعداد کلمات‌شان یکی است. این همان ضعفی است که در کلاس دیدید: BoW وجود کلمه را می‌بیند، ولی نقش و ترتیب را خوب نمایندگی نمی‌کند. نفی هم مثال خوبی است: دیدن کلمه not غیر از فهمیدن این است که not به کدام عبارت خورده.

بخش ۱ج. از شماره به embedding

این‌جا سه چیز را قاطی نکن:

- شماره کلمه فقط آدرس است. اگر فردا شماره‌ها عوض شوند، معنی کلمه عوض نشده است.
 - one-hot هویت را جدا می‌کند، ولی همه کلمه‌های متفاوت را «به یک اندازه نامربوط» می‌بیند.
 - embedding یک نقطه در فضا است. مدل می‌تواند یاد بگیرد hockey به puck نزدیک‌تر باشد تا به orbit.
- واژه‌نامه را از داده آموزش بساز. کلمات خیلی نادر را unk بگذار تا واژه‌نامه شلوغ نشود. طول سند را ثابت کن تا batch ساخت.

مدل این بخش را ساده بگیر: embedding هر توکن، بعد میانگین روی طول سند، بعد یک لایه برای دو کلاس. میانگین هنوز ترتیب را کامل نمی‌فهمد؛ عمداً همین را می‌خواهیم، چون قدم بعد Encoder است. جفت‌های skip-gram را هم از همین متن بساز، با پنجره ۲. لازم نیست Word2Vec کامل آموزش دهی. فقط ببین «مرکز / همسایه» یعنی چه. سیگنالش این است که کلمه‌هایی که کنار هم می‌آیند، باید بازنمایی نزدیک‌تری پیدا کنند.

یک نکته کنار دستت

شکل دسته شناسه‌ها این است: تعداد نمونه $\times$ طول دنباله. خروجی طبقه‌بند این است: تعداد نمونه $\times$ ۲ کلاس. اگر این شکل‌ها جور نبود، اول پدینگ و طول ثابت را چک کن.

بخش ۱د. Encoder کوچک

حالا توکن‌ها می‌توانند به هم نگاه کنند. Encoder برای طبقه‌بندی خبر باید کل سند را ببیند؛ پس ماسک علی‌نگذار. اگر آینده را ببندی، مدل را بی‌خودی محدود کرده‌ای. اینجا آینده معنا ندارد؛ کل خبر از قبل حاضر است. مدل تقریباً این قطعات را دارد: embedding توکن، embedding موقعیت، یک یا دو لایه TransformerEncoder، بعد میانگین خروجی روی طول، بعد لایه کلاس. موقعیت را برای این می‌گذاریم که توجه بفهمد کلمه اول با کلمه پنجم فرق دارد.

یک سند نمونه را رد کن و شکل را در چهار جا چاپ کن: شناسه، بعد از embedding، بعد از Encoder، بعد از لایه کلاس. طول دنباله و d_model وسط راه نباید بی‌دلیل عوض شوند.

یک نکته کنار دستت

batch_first=True را فراموش نکن. کلاس آماده PyTorch مجاز است. اگر norm_first=True بگذاری، همان ایده اتصال باقی‌مانده و نرمال‌سازی لایه در کلاس است.

بخش ۱ه. مقایسه

سه مدل را کنار هم بگذار و برای هر کدام یک جمله بنویس: چه چیزی را می‌بیند؟ سؤال‌های این بخش برای حفظ کردن نیست؛ برای این است که مسیر کلاس را به زبان خودت برگردانی. بدون موقعیت، جابه‌جا کردن توکن‌ها برای توجه تقریباً بی‌معنی می‌شود. نرمال‌سازی لایه روی ویژگی‌های هر توکن است، نه روی کل دسته. شبکه بازگشتی هم ترتیب را می‌فهمد، ولی مجبور است گام‌به‌گام جلو برود؛ گلوگاهش همین پردازش ترتیبی است. لازم نیست آن را پیاده کنی.

۲ پروژه ۲. قلم نمایشنامه‌نویس

در نوت‌بوک از عنوان «پروژه ۲» تا پایان بخش ۲ه.

اینجا چه می‌خواهیم؟

می‌خواهیم از متن خام، بدون لیبل انسانی، یاد بگیریم «بعدی چیست» و بعد از یک پیشوند چند توکن تولید کنیم. آخر کار باید فرق آموزش موازی و تولید توکن‌به‌توکن برایت روشن باشد.

داده Tiny Shakespeare است؛ متن واقعی و معروف مدل زبانی. کل فایل را نگیر. حدود ۱۲ تا ۲۰ هزار نویسه اول کافی است تا مدل روی لپ‌تاپ راه برود. قرار نیست شکسپیر واقعی بنویسی؛ قرار است مسیر pretraining را لمس کنی. اگر داندلود سخت بود، از Colab یا فایل محلی استفاده کن. یک آدرس شناخته‌شده:

<https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>

بخش ۲الف. متن و tokenizer

اینجا دو نگاه جدا داری و قاطی‌شان نکن.

نگاه اول مشاهده است: همان دو خط را بده به bert-base-uncased و gpt2. می‌خواهیم ببینی tokenizer معروف متن را چطور خرد می‌کند و آیا توکن خاص اول و آخر اضافه می‌کند یا نه. با این واژه‌نامه‌های بزرگ مدل خودت را آموزش نده؛ جدول embedding خیلی بزرگ می‌شود.

نگاه دوم آموزش مدل خودت است. توصیه می‌کنم واژه‌نامه را در سطح نویسه بسازی. آن وقت اندازه واژه‌نامه معمولاً چند ده نماد است و تمرین سخت نمی‌شود. نرمال‌سازی نمایشنامه را خیلی خشن نکن؛ خط جدید گاهی جزء سبک متن است. هر کاری کردی، در دو جمله بگو چرا.

بخش ۲ب. مثال next-token

لیبل را کسی نساخته. اگر دنباله این باشد:

t0 t1 t2 t3 t4

ورودی معمولاً t0 t1 t2 t3 است و هدف t1 t2 t3 t4. یعنی همه‌چیز یک خانه جابه‌جا شده. به این می‌گوییم خودنظارتی.

یک پنجره را از شماره به متن برگردان و با چشم ببین مدل باید چه چیزی را حدس بزند. اگر تعداد پنجره‌ها خیلی زیاد شد، چند صد تای اول برای این تمرین کافی است.

بخش ۲.ج. مدل decoder-style و ماسک علی

این مهم‌ترین فرق پروژه ۲ با پروژه ۱ است. وقتی می‌گوییم «توکن بعدی را بگو»، موقعیت ۳ نباید توکن ۴ و ۵ را ببیند. اگر ببیند، جواب را از آینده کپی می‌کند و یادگیری تقلب می‌شود. ماسک علی همین پرده است. برای طول ۵، ایده این است:

```
pos 1: sees 1
pos 2: sees 1, 2
pos 3: sees 1, 2, 3
```

در ماتریس کلاس، True یعنی توجه بسته است. مثلث بالایی آینده را می‌بندد. همان `torch.triu` با `diagonal=1`. مدل کوچک بساز: embedding توکن، embedding موقعیت، لایه Transformer، لایه خطی به اندازه وازه‌نامه. شکل logits باید شبیه `[batch, length, vocab]` باشد.

دو راه درست است: TransformerEncoder به همراه ماسک علی (همان الگوی کلاس)، یا TransformerDecoder بدون حافظه خارجی. هر کدام را انتخاب کردی، بگو چرا به آن می‌گوییم: decoder-style چون جریان اطلاعات مثل تولید متن است، نه مثل خواندن کل خبر. اگر کلاس‌های PyTorch را چاپ کنی، در Decoder معمولاً یک توجه اضافه برای نگاه به خروجی Encoder می‌بینی؛ اسمش cross-attention است. در این پروژه اگر Decoder بدون حافظه خارجی استفاده کنی، همان بخش ممکن است بی‌استفاده بماند و اشکالی ندارد.

بخش ۲.د. آموزش و تولید

چند ده step کافی است. loss لازم نیست صفر شود؛ فقط باید نسبتاً پایین بیاید. اگر نیامد، اول ماسک، شکل logits، و جفت ورودی/هدف را چک کن. تولید با آموزش فرق دارد. در آموزش همه موقعیت‌های ورودی از قبل حاضرند؛ توجه می‌تواند موازی حساب شود و ماسک جلوی تقلب را می‌گیرد. در تولید، توکن بعدی هنوز وجود ندارد. یکی را پیش‌بینی می‌کنی، به ورودی اضافه می‌کنی، دوباره پیش‌بینی می‌کنی. برای شروع `argmax` کافی است. از ROME0 یا First Citizen: شروع کن. خروجی ضعیف یا تکراری طبیعی است. بگو چه الگویی را تا حدی گرفته (مثلاً فاصله، حروف بزرگ، نام‌ها) و چه چیزی را هنوز بلد نیست.

بخش ۲.ه. یک آزمایش کوچک

یکی از سه کار نوت‌بوک را انتخاب کن. هدف این است که یک فرض را خراب کنی و ببینی چه می‌شود. بدون ماسک علی، مدل در آموزش می‌تواند آینده را ببیند؛ برای پیش‌بینی توکن بعدی این نشت اطلاعات است. بدون موقعیت، ترتیب ضعیف می‌شود. فرق آموزش موازی و تولید ترتیبی را هم اگر با شاهد از کد بگویی، همان درس جلسه است.

دو سؤال آخر را جدی بگیر، چون دو پروژه را به هم وصل می‌کنند: پروژه ۱ کل خبر را دارد، پس Encoder آزاد است همه را ببیند. پروژه ۲ باید بعدی را بسازد، پس آینده باید پنهان بماند. pretraining این پروژه از خود متن یاد می‌گیرد؛ fine-tune روی خبر یعنی همان وزن‌ها را برای یک وظیفه دارای لیبل، مثل تشخیص موضوع، تنظیم کنیم.